

Universidade do Minho

Escola de Engenharia

Licenciatura em Engenharia Informática

Mestrado Integrado em Engenharia Informática

Unidade Curricular de Computação Gráfica

Ano Letivo de 2025/2026

Fase 1: Primitivas Gráficas

Jorge Rafael Machado Fernandes
A104168

Diogo Teixeira Fernandes
A104260

Filipe Teixeira Viana
A104361

Data da Receção	
Responsável	
Avaliação	
Observações	

Fase 1: Primitivas Gráficas

fevereiro, 2026

Índice

1. Introdução	1
2. Generator	2
2.1. Funcionamento do Generator	2
2.2. Formato de Ficheiro de Saída .3d	3
2.3. Organização e Orientação dos Triângulos	3
2.3.1. Orientação dos Vértices (Winding Order)	3
2.4. Convenção de Nomenclatura	3
2.5. Implementação das Primitivas	4
2.5.1. Superfície Plana (Plane)	4
2.5.1.1. Centralização e Grelha de Subdivisão	4
2.5.1.2. Triangulação	4
2.5.2. Caixa (Box)	5
2.5.3. Esfera (Sphere)	6
2.5.3.1. Cálculo das coordenadas	6
2.5.3.2. Conversão para Coordenadas Cartesianas	6
2.5.3.3. Triangulação da superfície	6
2.5.4. Cone	7
2.5.4.1. Cálculo da Geometria	7
2.5.4.2. Coordenadas dos pontos	7
2.5.4.3. Processo de Geração	8
3. Engine	9
3.1. Renderização da Cena	9
3.1.1. Leitura e Interpretação do XML	9
3.1.1.1. Configuration	9
3.1.1.2. Window	10
3.1.1.3. Camera	10
3.1.2. Renderização dos Modelos	10
3.1.2.1. Leitura de ficheiros e parsing	10
3.1.2.2. Armazenamento e Desenho	10
3.2. Exploração da Cena	10
3.2.1. Movimento Orbital	10
3.2.2. Zoom	11
3.2.3. Outros	11
4. Utilitários	12
4.1. Biblioteca	12
4.2. Estruturas de Dados Partilhadas	12

Lista de Figuras

Figura 1	Plano com 4 de largura e 6 divisões	4
Figura 2	Caixa com 2 de largura e 3 divisões	5
Figura 3	Esfera com 1 de raio, 10 slices e 10 stacks	7
Figura 4	Cone com 1 de raio, altura 2, 4 slices e 3 stacks	8
Figura 5	Movimento orbital da câmara	11

1. Introdução

Este relatório foi realizado na Unidade Curricular de Computação Gráfica no ano letivo de 2025/2026 e aborda do desenvolvimento desta primeira fase do trabalho prático, que implementa 2 componentes essenciais, o **Generator** e a **Engine**.

O trabalho desenvolvido consiste num pequeno motor gráfico dividido em duas componentes principais: um **Generator** que produz ficheiros de modelos geométricos e um **Engine** que os carrega e renderiza em OpenGL.

A ideia geral do projeto é permitir que um utilizador crie primitivas tridimensionais parametrizadas, as quais depois podem ser visualizadas e manipuladas no visualizador desenvolvido.

2. Generator

O **Generator** trata-se de um programa de linha de comandos cuja responsabilidade primária é criar primitivas geométricas tridimensionais (plano, esfera, caixa ou cone) seguindo os parâmetros informados pelo utilizador. Este módulo funciona como ferramenta de suporte ao motor gráfico, gerando os ficheiros de modelos que serão posteriormente carregados e renderizados pelo Engine.

2.1. Funcionamento do Generator

O **Generator** foi concebido como uma ferramenta de suporte operando exclusivamente através da linha de comandos, garantindo portabilidade e facilidade de integração em pipelines automatizados. A sua responsabilidade é a geração de superfícies geométricas utilizando exclusivamente triângulos como primitiva elementar.

O fluxo lógico implementado no main.cpp segue a seguinte sequência:

1. **Validação de argumentos:** o programa valida o número de argumentos fornecidos (mínimo 5, variável conforme o tipo de figura);
2. **Parsing de entrada:** os argumentos da shell são convertidos em tipos numéricos computáveis (float para dimensões, int para divisões);
3. **Redirecionamento para o gerador específico:** consoante o primeiro argumento (plane, sphere, box ou cone), o fluxo é redirecionado para a função correspondente (generatePlane, generateSphere, generateBox, generateCone);
4. **Computação geométrica:** cada função calcula as coordenadas 3D dos vértices necessários, respeitando parâmetros como raio, altura, número de slices/stacks ou divisões;
5. **Persistência:** o vetor de pontos resultante é serializado num ficheiro .3d através da função saveToFile (módulo utils.cpp).

Esta arquitetura modular facilita tanto o debugging como a expansão futura; uma nova primitiva requer apenas adicionar uma nova função e integra-la no dispatcher do main.

2.2. Formato de Ficheiro de Saída .3d

Os ficheiros **.3d** gerados seguem uma estrutura textual simples. A primeira linha indica o número total de vértices do modelo, sendo as linhas seguintes compostas pelas coordenadas de cada vértice no formato *xyz*, separadas por espaço. Cada três vértices consecutivos formam um triângulo da primitiva, tornando o formato diretamente compatível com a renderização via **glDrawArrays** do OpenGL. A título de exemplo, um plano simples composto por dois triângulos seria representado da seguinte forma:

```
6
-1 1 0
-1 -1 0
1 1 0
1 -1 0
-1 -1 0
1 -1 0
```

2.3. Organização e Orientação dos Triângulos

Em todas as primitivas geradas, os vértices de cada triângulo são definidos seguindo a ordem anti-horária, garantindo que as faces exteriores fiquem corretamente orientadas. Esta abordagem é fundamental para que o mecanismo de back-face culling do OpenGL, ativado através de `glEnable(GL_CULL_FACE)`, possa descartar automaticamente as faces traseiras, tornando a renderização mais eficiente e livre de artefactos visuais.

2.3.1. Orientação dos Vértices (Winding Order)

- Os vértices de cada triângulo são ordenados no sentido anti-horário (Counter-Clockwise, CCW) quando observados do ponto de vista exterior da figura.
- Esta decisão de design é fundamental para o funcionamento correto do Face Culling no pipeline gráfico OpenGL, onde as normais implícitas (calculadas pelo produto vetorial dos vértices) apontam para o exterior do volume.
- O plano respeita CCW em relação ao normal ascendente (eixo Y positivo); a esfera garante CCW em relação ao vetor radial; o cone e a caixa mantêm consistência face a face.

2.4. Convenção de Nomenclatura

O nome do ficheiro de saída é definido pelo utilizador e passado como último argumento na linha de comando. Por exemplo, ao executar:

```
generator cone 1 2 4 3 cone_1_2_4_3.3d
```

O ficheiro gerado terá o nome `cone_1_2_4_3.3d`. Esta abordagem oferece total liberdade ao utilizador para organizar e identificar os modelos gerados de acordo com os parâmetros utilizados ou com qualquer outra convenção que considere adequada.

2.5. Implementação das Primitivas

Nesta secção, detalha-se a lógica geométrica e os algoritmos de geração para cada uma das formas implementadas.

Todas as primitivas são calculadas tendo como referência a origem (0, 0, 0) e a sua geração é parametrizada para permitir diferentes níveis de detalhe (resolução da malha).

2.5.1. Superfície Plana (Plane)

Para a geração do plano, são necessários dois parâmetros:

- **Largura:** a largura total do plano;
- **Divisões:** o número de subdivisões em cada eixo.

2.5.1.1. Centralização e Grelha de Subdivisão

O plano é centrado na origem **(0,0,0)** dividindo a largura por 2, obtendo **halfSize**. Sendo L o comprimento e do número de divisões, a distância entre vértices consecutivos é dado por "**step**" = L / d .

2.5.1.2. Triangulação

Cada célula da grelha é definida por 4 vértices com **y=0**, calculados a partir dos índices (i,j):

- $x1 = -halfSize + i \times step$
- $z1 = -halfSize + j \times step$
- $x2 = x1 + step$
- $z2 = z1 + step$

Cada célula origina dois triângulos:

- **Triângulo 1:** (P1,P3,P4)
- **Triângulo 2:** (P1,P4,P2)

Na figura abaixo é possível observar o plano gerado, com 4 unidades de largura e 6 divisões.

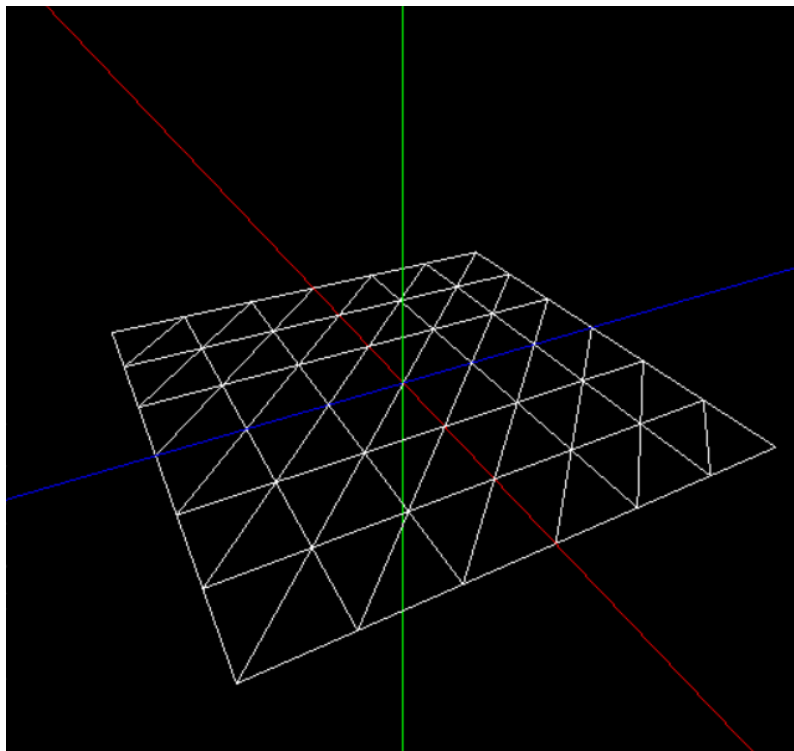


Figura 1: Plano com 4 de largura e 6 divisões

2.5.2. Caixa (Box)

A caixa está centrada na origem e é definida por uma largura e um número de divisões. Tal como no plano, calcula-se **halfSize** e **step**:

$$\text{halfSize} = \frac{\text{length}}{2}$$

$$\text{step} = \frac{\text{length}}{\text{divisions}}$$

São geradas seis faces independentes, cada uma com dois triângulos por célula da grelha:

- **Base inferior** ($y = -\text{halfSize}$) e **Base superior** ($y = +\text{halfSize}$): pontos distribuídos no plano horizontal;
- **Face frontal** ($z = +\text{halfSize}$) e **Face traseira** ($z = -\text{halfSize}$): pontos ao longo dos eixos X e Y;
- **Face direita** ($x = +\text{halfSize}$) e **Face esquerda** ($x = -\text{halfSize}$): pontos ao longo dos eixos Z e Y.

Na figura abaixo é possível observar a caixa gerada, com 2 de largura e 3 divisões.

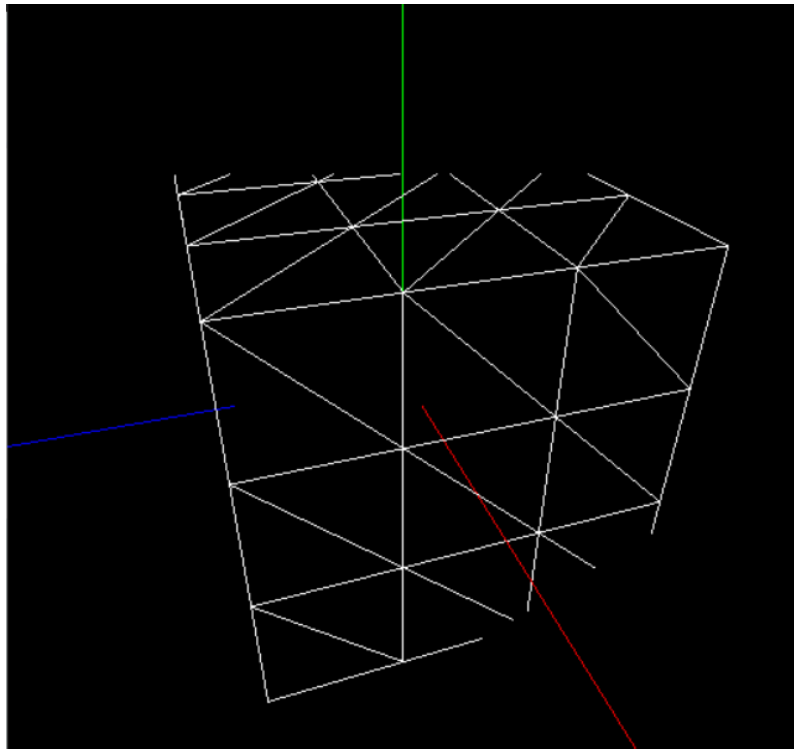


Figura 2: Caixa com 2 de largura e 3 divisões

2.5.3. Esfera (Sphere)

Para a construção da esfera, são necessários três parâmetros: o raio, o número de slices e o número de stacks. Estes controlam a precisão e a segmentação da malha esférica, permitindo ajustar o nível de detalhe da figura final.

A superfície é dividida em slices (divisões longitudinais) e stacks (divisões latitudinais), formando uma grelha de quadriláteros que é triangulada para renderização.

2.5.3.1. Cálculo das coordenadas

Para cada combinação de slice e stack, são calculados dois ângulos de latitude (φ) e dois de longitude (θ):

$$\Delta\varphi = \frac{\pi}{\text{stacks}}$$

$$\Delta\theta = \frac{2\pi}{\text{slices}}$$

Para cada iteração i (stack) e j (slice):

$$\varphi_1 = -\frac{\pi}{2} + i \cdot \Delta\varphi$$

$$\varphi_2 = -\frac{\pi}{2} + (i + 1) \cdot \Delta\varphi$$

$$\theta_1 = j \cdot \Delta\theta$$

$$\theta_2 = (j + 1) \cdot \Delta\theta$$

2.5.3.2. Conversão para Coordenadas Cartesianas

Cada ponto da malha é calculado através das seguintes fórmulas:

$$x = r \cdot \cos(\Phi) \cdot \sin(\theta)$$

$$y = r \cdot \sin(\Phi)$$

$$z = r \cdot \cos(\Phi) \cdot \cos(\theta)$$

2.5.3.3. Triangulação da superfície

Cada célula da grelha é dividida em dois triângulos:

- **Triângulo 1:** (P1,P2,P3)
- **Triângulo 2:** (P2,P4,P3)

Na figura abaixo é possível observar a esfera gerada, com raio 1, 10 slices e 10 stacks:

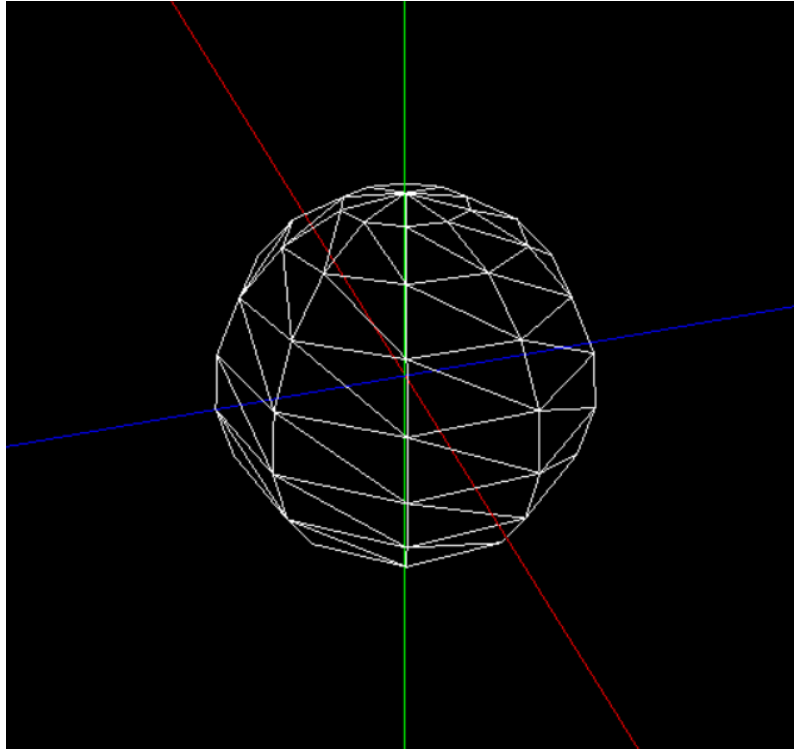


Figura 3: Esfera com 1 de raio, 10 slices e 10 stacks

2.5.4. Cone

O cone é parametrizado por quatro valores:

- **Raio da base;**
- **Altura total;**
- **Slices:** divisões radiais que subdividem a circunferência da base em fatias angulares iguais;
- **Stacks:** divisões verticais ao longo da altura, formando anéis de raio decrescente até ao ápice.

2.5.4.1. Cálculo da Geometria

O ângulo de cada slice e a altura de cada stack são calculados por:

$$\alpha = \frac{2\pi}{\text{slices}}$$

$$\text{stackHeight} = \frac{\text{altura}}{\text{stacks}}$$

2.5.4.2. Coordenadas dos pontos

Para cada stack, são calculados dois raios consecutivos que diminuem linearmente:

$$r_{\text{atual}} = \text{radius} - \text{stack} \cdot \frac{\text{radius}}{\text{stacks}}$$

$$r_{\text{seguinte}} = \text{radius} - (\text{stack} + 1) \cdot \frac{\text{radius}}{\text{stacks}}$$

Os pontos laterais de cada anel são calculados com base no ângulo de cada slice:

$$x = r \cdot \sin(\theta)$$

$$z = r \cdot \cos(\theta)$$

$$y = \text{stack} \cdot \text{stackHeight}$$

2.5.4.3. Processo de Geração

A superfície lateral é gerada com dois ciclos aninhados — um para as slices e um para as stacks. Para cada par, são gerados dois triângulos que formam um quadrilátero da malha. A base do cone é gerada como um leque de triângulos, todos com um vértice no centro **(0,0,0)** e os outros dois em pontos consecutivos da circunferência. A progressão decrescente do raio garante que todos os triângulos da última stack convergem para o ápice em **(0,h,0)**.

Na figura abaixo é possível observar o cone gerado, com raio 1, altura 2, 4 slices e 3 stacks:

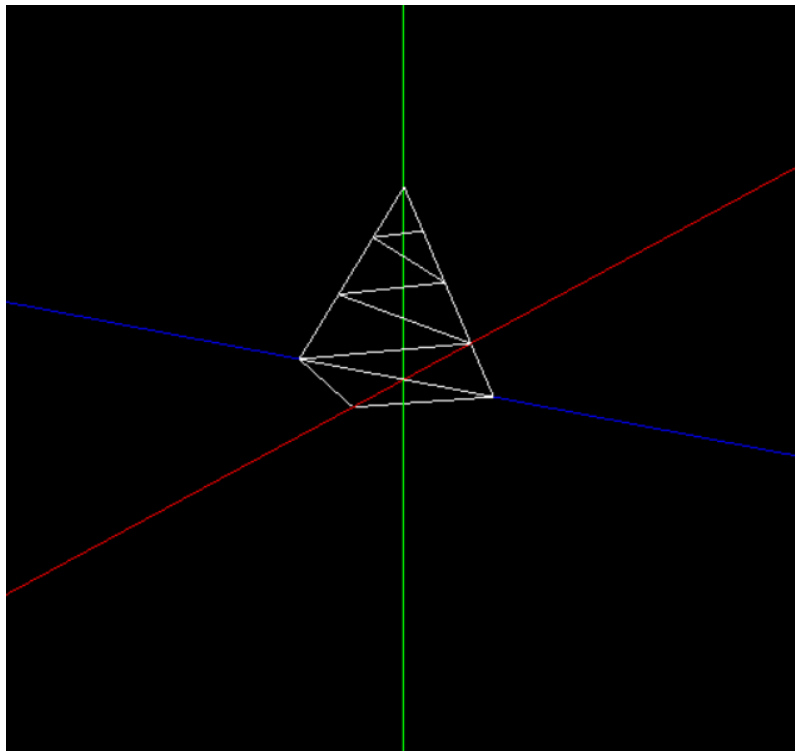


Figura 4: Cone com 1 de raio, altura 2, 4 slices e 3 stacks

3. Engine

O **Engine** desenvolvido permite a visualização das primitivas através da leitura e interpretação de ficheiros XML com as configurações iniciais.

O ficheiro XML fornecido deve estar corretamente estruturado, contendo as seguintes informações:

- **Dimensões da janela** (window), definidas pelos atributos width e height;
- **Parâmetros da câmara** (camera), incluindo:
 - Posição da câmara (position), determinada pelos atributos x, y e z;
 - Ponto de observação (lookAt), que indica para onde a câmara está orientada;
 - Orientação da câmara (up), especificando o vetor que representa a direção “para cima”;
 - Projeção (projection), definida pelo ângulo de visão (fov), distância mínima (near) e máxima (far).
- **Grupo de modelos** (group), onde cada modelo é representado por uma referência ao respetivo ficheiro .3d.

Para além das configurações definidas inicialmente na execução do Engine, a aplicação permite atualizar a posição da câmara, bem como aplicar zoom. É também possível alterar o estilo de visualização da primitiva entre linhas, pontos ou preenchido, além de ativar ou desativar a visualização dos eixos da cena, proporcionando maior flexibilidade na exploração do ambiente 3D.

3.1. Renderização da Cena

3.1.1. Leitura e Interpretação do XML

A configuração do motor gráfico é carregada a partir de um ficheiro XML. Esse processo é realizado pela função **setupConfig**, que recebe o caminho do ficheiro e faz as verificações necessárias para garantir uma correta extração. A função **parseConfig** é utilizada para interpretar o ficheiro, utilizando a biblioteca **RapidXML** para processar os dados e armazená-los em estruturas apropriadas.

3.1.1.1. Configuration

A classe **Configuration** serve como estrutura central para guardar toda a informação extraída do ficheiro XML, reunindo num único objeto as dimensões da janela, os parâmetros da câmara e os modelos a renderizar.

Para realizar este processo, a função **parseConfig** segue três etapas principais:

- **Leitura do ficheiro XML:** O ficheiro é aberto e o seu conteúdo é carregado integralmente para uma string, que será posteriormente processada.
- **Interpretação com RapidXML:** A string com o conteúdo XML é analisada pela biblioteca RapidXML, que percorre os diferentes nós do documento para identificar e extrair os dados de configuração.
- **Armazenamento nas estruturas de dados:** Com os dados extraídos, são criadas e inicializadas as instâncias das classes Window, Camera e a lista de modelos associados à cena.

3.1.1.2. Window

O nó **window** é responsável por definir as dimensões da janela de renderização. A partir deste nó, são lidos os atributos **width** e **height**, que são posteriormente convertidos para valores inteiros e armazenados na classe Window.

3.1.1.3. Camera

As definições da câmera são obtidas a partir do nó camera, que agrupa os parâmetros necessários para posicionar e orientar a visualização na cena 3D. Entre estes parâmetros encontram-se a posição **position**, **lookAt**, **up**, **fov**, **nearP** e **farP**. Estes valores são utilizados para configurar corretamente a projeção perspetiva da cena.

3.1.2. Renderização dos Modelos

O sistema de renderização de modelos 3D implementado é responsável por carregar ficheiros com modelos tridimensionais, processá-los e desenhá-los no ecrã utilizando a biblioteca OpenGL. Os modelos são carregados a partir de ficheiros no formato **.3d**, gerados pelo **Generator**, e armazenados em memória como listas de vértices para posterior renderização.

3.1.2.1. Leitura de ficheiros e parsing

A leitura dos modelos 3D é realizada pela função `parse3Dfile()`, que carrega diretamente uma lista de pontos a partir de ficheiros no formato **.3d**. A primeira linha do ficheiro contém o número total de vértices, e as linhas seguintes contêm as coordenadas x, y, z de cada ponto. Cada conjunto de três pontos consecutivos forma um triângulo, que será posteriormente desenhado pelo engine.

3.1.2.2. Armazenamento e Desenho

Depois do parsing, os vértices de cada modelo ficam guardados numa lista global, implementada como um **std::vector** de listas de pontos. No momento de renderizar a cena, cada modelo é processado pela função **drawTriangles()**, que itera sobre a lista de pontos em grupos de três, enviando as coordenadas de cada vértice para a GPU através da função **glVertex3f()**, pertencente à biblioteca OpenGL.

3.2. Exploração da Cena

De modo a que o utilizador consiga explorar o objeto 3D gerado, foi implementada uma **câmara dinâmica**, permitindo uma observação geral da cena. A câmara pode rodar, aproximar-se e afastar-se do objeto, proporcionando uma experiência interativa ao utilizador.

3.2.1. Movimento Orbital

A câmara segue um movimento orbital em torno da origem do cenário, permitindo ao utilizador observar o modelo tridimensional de diferentes ângulos. A posição da câmara é calculada com base em dois ângulos, **alpha** e **beta**, e no raio da esfera imaginária **camRadius**:

```
camX = camRadius * cos(beta) * sin(alpha);  
camY = camRadius * sin(beta);  
camZ = camRadius * cos(beta) * cos(alpha);
```

O controlo é feito através das setas do teclado:

- ← / → ajustam o ângulo horizontal (**alpha**), girando a câmara em torno do eixo Y.
- ↑ / ↓ ajustam o ângulo vertical (**beta**), movimentando a câmara para cima ou para baixo.

Dessa forma, o utilizador pode orbitar livremente em torno do objeto renderizado sem alterar a sua distância.

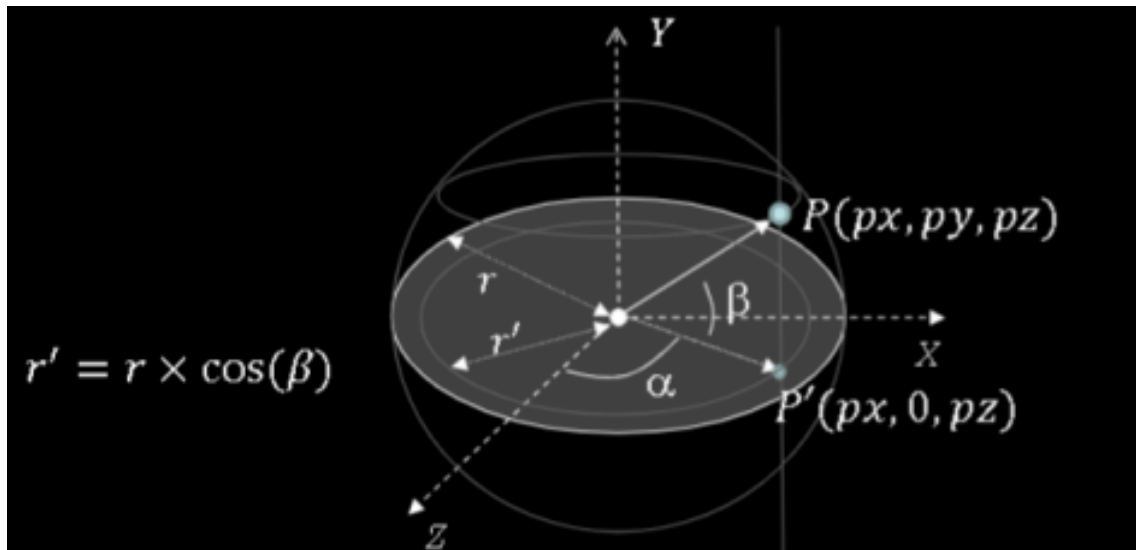


Figura 5: Movimento orbital da câmara

3.2.2. Zoom

- Tecla **o**: Afasta a câmara da origem (Zoom Out).
- Tecla **i**: Aproxima a câmara da origem (Zoom In).

3.2.3. Outros

- Tecla **a**: Ativa ou desativa os eixos de referência.
- Tecla **r**: Reinicia a posição da câmara para a configuração inicial do XML.
- Tecla **m**: Alterna entre os modos de renderização (wireframe, pontos e preenchido).

4. Utilitários

4.1. Biblioteca

Para suportar a leitura e interpretação dos ficheiros de configuração, foi integrada no projeto a biblioteca **RapidXML**. Esta biblioteca, especializada no processamento de ficheiros XML, permitiu extrair de forma eficiente os parâmetros da câmara, as dimensões da janela e os modelos a carregar, sendo uma solução leve e de fácil integração em projetos C++.

4.2. Estruturas de Dados Partilhadas

De forma a promover a reutilização de código entre os diferentes módulos do projeto, foi implementada a classe **Point**, que encapsula as coordenadas tridimensionais (**x**, **y**, **z**) de um ponto no espaço 3D. Esta classe serve de base à representação de todas as primitivas geométricas, cujos modelos são internamente descritos como coleções de objetos **Point** armazenados em vetores.